

Pipeline Hw practice

1. Write an assembly routine that divides two numbers.

DIV: # a0=a0/a1

addi sp,sp,-4

sw s0, 0(sp)

li s0, 0

blt a0, a1, END

SUB: sub a0, a0, a1 addi s0, s0, 1 bgt a0, a1, SUB

END: #a0=remainder mv a0, s0 # value

lw s0, 0(sp)

addi sp,sp,4

ret

2. Add NOP instructions to the code below so that it will run correctly on a pipeline that does not handle data hazards

addi x11, x12, 5

NOP

NOP

add x13, x11, x12

NOP

addi x14, x11, 15

add x15,x13,x12

So....7 clock cycles. Can you reorder instructions to avoid NOPs?

3. Consider the following loop:

LOOP: lw x10, 0(x13)

lw x11, 8(x13)

add x12, x10, x11 + 2 cycle stall

subi x13, x13, 16 + 1 cycle stall

bnez x12, LOOP + 2 cycle stall

How many cycles do the first 3 loop iterations take? Assumptions?

Assume Pipeline already full, only using NOP, 24 cycles?

(Assume the pipeline inserts stalls where needed)

Control hazards

w/o forwarding?

5 cycles + 2 cycles + 1 cycle + 2 cycles = 10 cycles

Three iterations = 30 cycles

w/ forwarding?

5 cycles + 1 cycle (load-use hazard) + 2 cycle = 8 cycles

24 cycles for 3 iterations

w/ delay-branch (1-delay slot) + forwarding?

LOOP: lw x10, 0(x13)

lw x11, 8(x13)

add x12, x10, x11 + 1 cycle stall

bnez x12, LOOP + 1 cycle stall

subi x13, x13, 16

5 + 1 + 1 = 7 cycles /iteration

21 cycles for 3 iterations

4. Assume no forwarding, how many cycles are needed for the following code snippet? With forwarding?

add x15, x12, x11

lw x13, 4(x15) +2

lw x12, 0(x2)

or x13, x15, x13 +1

sw x13, 0(x15) +2

w/o forwarding: 5 clock cycles + 5 stall cycles = 10 cycles

w/ forwarding: 5 clock cycles = 5 clock cycles. 2X speedup

5. Write the assembly code the following code. How many cycles does it take? Can you rewrite to make it run faster?

```
for (i=1, s=0 ; i < 100 ; i++) {  
    s += i*i;  
}
```

Don't have to run for $i = 0$ since s is already 0. Yes!

You can assume you have a multiply instruction for this problem :) yay!

You can also use loop unrolling to get rid of branches. This will require more memory.

```
Li t0, 1 # i
```

```
Li a0, 0 # s
```

```
Li t3, 100
```

```
for:
```

```
mv t1, a0 #saves s value
```

```
mv a1, t0 #set up multiplication
```

```
mv a0, t0
```

```
call mult #uses a0, a1 for multiplication, returns on a0
```

```
add a0, a0, t1 # s += i*i
```

```
addi t0, t0, 1 #increment counter
```

```
blt t0, t3, for #branch if less than 100
```